МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования

**«Белорусский государственный университет информатики и радиоэлектроники»**

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

# ОТЧЕТ

по лабораторной работе №2

по дисциплине

«Информационные и технические средства защиты авторских прав»

«Криптосистема с открытым ключом RSA»

**Выполнил:**
Лебедевич Артем Владимирович
магистрант группы 556301

**Преподаватель:**
Ярмолик Вячеслав Николаевич
доктор технических наук
профессор кафедры ПОИТ

Минск, 2026 г.

# СОДЕРЖАНИЕ

1 Цель работы ..... 1

2 Теоретические сведения ..... 2
2.1 Криптосистемы с открытым ключом ..... 3
2.2 Генерация ключей RSA ..... 3
2.3 Шифрование и дешифрование ..... 3
2.4 Расширенный алгоритм Евклида ..... 4
2.5 Быстрое возведение в степень ..... 4
2.6 Пример из методического пособия ..... 5
2.7 Криптостойкость RSA ..... 5

3 Реализация ..... 5
3.1 Структура программы ..... 6
3.2 Ключевые фрагменты кода ..... 6
3.3 Интерфейс командной строки ..... 8
3.4 Графическое приложение ..... 8

4 Результаты работы программы ..... 8

5 Анализ результатов ..... 10

Заключение ..... 12

Список использованных источников ..... 13

## 1 ЦЕЛЬ РАБОТЫ

Изучить принципы построения криптосистем с открытым ключом и алгоритм RSA. В соответствии с вариантом №1 (нечётный номер в списке группы) реализовать шифратор и дешифратор RSA, использующие алгоритм быстрого возведения в степень по модулю, а также вычисление пары ключей по заданным значениям $p$, $q$ и $e$ с помощью расширенного алгоритма Евклида. Проверить корректность реализации на примере из методического пособия.

## 2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

### 2.1 Криптосистемы с открытым ключом

В симметричных шифрах, рассмотренных в лабораторной работе №1, отправитель и получатель используют один и тот же секретный ключ, поэтому до начала переписки ключ необходимо передать по защищённому каналу. Криптосистемы с открытым ключом (асимметричные) устраняют этот недостаток: каждый участник генерирует пару ключей — открытый $K_o$ и секретный $K_c$. Данные, зашифрованные открытым ключом, можно расшифровать только секретным, причём вычислить $K_c$ по известному $K_o$ практически невозможно. Следовательно, открытый ключ может распространяться по любым открытым каналам.

Основой таких систем служат односторонние функции с секретом: функции, которые легко вычислить, трудно обратить, но легко обратить при знании дополнительной секретной информации. Для RSA такой функцией является возведение в степень по модулю составного числа $r = p \cdot q$, а секретом — разложение $r$ на простые множители.

### 2.2 Генерация ключей RSA

Пара ключей RSA создаётся следующим образом:

1. Выбираются два секретных простых числа $p$ и $q$, близких по величине.
2. Вычисляется модуль $r = p \cdot q$.
3. Вычисляется функция Эйлера $\varphi(r) = (p-1)(q-1)$ — количество чисел, меньших $r$ и взаимно простых с ним.
4. Выбирается открытая экспонента $e$: $1 < e < \varphi(r)$, $\gcd(e, \varphi(r)) = 1$.
5. Вычисляется секретная экспонента $d$, удовлетворяющая условию

$$e \cdot d \bmod \varphi(r) = 1, \tag{2.1}$$

то есть $d$ — мультипликативное обратное к $e$ по модулю $\varphi(r)$.

Открытым ключом является пара $K_o = (e, r)$, секретным — $K_c = (d, r)$. Значения $p$, $q$, $\varphi(r)$ и $d$ хранятся в секрете.

### 2.3 Шифрование и дешифрование

Сообщение $M$ разбивается на блоки $m_1, m_2, \ldots$, каждый из которых представляет собой число из диапазона $0 \ldots r-1$. Шифрование открытым ключом и дешифрование секретным выполняются по формулам

$$c_i = m_i^e \bmod r, \qquad m_i = c_i^d \bmod r. \tag{2.2}$$

Корректность следует из теоремы Эйлера. Поскольку $ed = 1 + k\varphi(r)$,

для $m$, взаимно простого с $r$, имеем

$$c^d = m^{ed} = m \cdot \left(m^{\varphi(r)}\right)^k \equiv m \cdot 1^k = m \pmod{r}. \tag{2.3}$$

Для $m$, кратного $p$ или $q$, равенство доказывается отдельно по каждому модулю и собирается по китайской теореме об остатках, поэтому дешифрование корректно для любого $m < r$.

### 2.4 Расширенный алгоритм Евклида

Секретная экспонента находится расширенным алгоритмом Евклида (EUCLIDEX), который одновременно с $\gcd(a, b)$ вычисляет коэффициенты $x$, $y$ соотношения Безу

$$x \cdot a + y \cdot b = \gcd(a, b). \tag{2.4}$$

Если положить $a = \varphi(r)$, $b = e$ и числа взаимно просты, то $y \cdot e \equiv 1 \pmod{\varphi(r)}$, то есть $y$ и есть искомое $d$. Отрицательное $y$ приводится в диапазон прибавлением $\varphi(r)$. Алгоритм хранит две последние тройки $(d, x, y)$ и на каждом шаге вычисляет новую по частному $q = d_0 \operatorname{div} d_1$. Ход вычислений для примера из пособия ($\varphi(r) = 20$, $e = 7$) приведён в таблице 2.1.

Таблица 2.1 – Расширенный алгоритм Евклида для $a = 20$, $b = 7$

| Шаг | $q$ | $d$ | $x$ | $y$ |
|---|---|---|---|---|
| 0 | — | 20 | 1 | 0 |
| 1 | — | 7 | 0 | 1 |
| 2 | 2 | 6 | 1 | $-2$ |
| 3 | 1 | 1 | $-1$ | 3 |

Цикл останавливается при $d_1 = 1$: $(-1) \cdot 20 + 3 \cdot 7 = 1$, откуда $d = 3$. Число итераций алгоритма Евклида не превышает $O(\log \varphi(r))$, поэтому вычисление ключа выполняется мгновенно даже для 2048-битных модулей.

### 2.5 Быстрое возведение в степень

Непосредственное вычисление $m^e$ невозможно: для 1024-битных чисел результат содержал бы порядка $2^{1024}$ цифр. Алгоритм быстрого возведения в степень (fast_exp) использует двоичное разложение показателя: пока показатель чётный, он делится пополам, а основание возводится в квадрат; при нечётном показателе из него вычитается единица и текущее основание домножается к результату. Все операции выполняются по модулю $n$, поэто-

му промежуточные значения не превосходят $n^2$. Количество умножений не превышает $2\log_2 z$.

Ход вычисления $4^7 \bmod 33$ приведён в таблице 2.2. Для показателя $7 = 111_2$ выигрыш невелик, но для 1024-битного показателя требуется порядка 1500 умножений вместо $2^{1024}$.

Таблица 2.2 – Быстрое возведение в степень $4^7 \bmod 33$

| $z_1$ | $a_1$ | $x$ | Действие |
|---|---|---|---|
| 7 | 4 | 1 | начальные значения |
| 6 | 4 | 4 | $z_1$ нечётно: $x = 1 \cdot 4$ |
| 2 | 16 | 31 | $z_1 = 3$, $a_1 = 4^2$; затем $x = 4 \cdot 16 \bmod 33$ |
| 0 | 25 | 16 | $z_1 = 1$, $a_1 = 16^2 \bmod 33$; затем $x = 31 \cdot 25 \bmod 33$ |

### 2.6 Пример из методического пособия

Для $p = 3$, $q = 11$ получаем $r = 33$, $\varphi(r) = 20$; при $e = 7$ расширенный алгоритм Евклида даёт $d = 3$. Буквы английского алфавита кодируются числами $A = 2, B = 3, \ldots, Z = 27$ (значения 0 и 1 пропускаются, поскольку $0^e = 0$ и $1^e = 1$ при любом ключе остались бы незашифрованными). Сообщение $M =$ «CAT» принимает вид $\{4, 2, 21\}$:

$$c_1 = 4^7 \bmod 33 = 16, \quad c_2 = 2^7 \bmod 33 = 29, \quad c_3 = 21^7 \bmod 33 = 21.$$

Дешифрование секретным ключом $K_c = (3, 33)$ восстанавливает исходные числа: $16^3 \bmod 33 = 4$, $29^3 \bmod 33 = 2$, $21^3 \bmod 33 = 21$.

Заметим, что блок $m_3 = 21$ перешёл сам в себя. Такие неподвижные точки ($m^e \equiv m$) неизбежны в RSA; при малом модуле их доля заметна, при модуле в тысячи бит вероятность попасть в неподвижную точку пренебрежимо мала.

### 2.7 Криптостойкость RSA

Стойкость RSA основана на двух трудных задачах. Восстановить $m$ по $c$ без ключа означает извлечь корень степени $e$ по модулю составного числа. Вычислить $d$ по открытому ключу означает найти $\varphi(r)$, что эквивалентно разложению $r$ на множители. Ни для одной из этих задач не известно полиномиального алгоритма. Лучший известный метод факторизации — общий метод решета числового поля — имеет субэкспоненциальную сложность, и рекордная разложенная задача RSA-250 содержит 829 бит. В настоящее время рекомендуемая длина модуля составляет не менее 2048 бит.

# 3 РЕАЛИЗАЦИЯ

## 3.1 Структура программы

Программа написана на языке Python 3 без внешних зависимостей. Код второй лабораторной работы находится в отдельном пакете `lab2`, общие теоретико-числовые функции вынесены в `helpers/numtheory.py`, поскольку они используются и в лабораторной работе №3 (листинг 3.1).

Листинг 3.1 – Структура программы

```
main.py                CLI: команда rsa (keys, enc, dec), demo 2, gui 2
configs/cfg.py         p, q, e по умолчанию (пример из пособия)
helpers/numtheory.py   fast_exp, euclid_ex, inverse, тест Миллера-Рабина
lab2/rsa.py            ключи, шифрование, дешифрование, кодирование текста
lab2/runner.py         прогон примеров пособия и самопроверка
lab2/gui.py            окно: ключи, шифрование, пошаговый разбор
```

## 3.2 Ключевые фрагменты кода

Расширенный алгоритм Евклида (листинг 3.2) повторяет псевдокод EUCLIDEX из пособия. Необязательный параметр `trace` собирает строки таблицы 2.1, которые затем выводятся в демонстрации и в окне программы.

Листинг 3.2 – Расширенный алгоритм Евклида

```
def euclid_ex(a, b, trace=None):
    d0, d1 = a, b
    x0, x1 = 1, 0
    y0, y1 = 0, 1
    while d1 > 1:
        q, d2 = divmod(d0, d1)
        x2, y2 = x0 - q * x1, y0 - q * y1
        d0, d1, x0, x1, y0, y1 = d1, d2, x1, x2, y1, y2
        if trace is not None:
            trace.append(EuclidStep(q, d1, x1, y1))
    if d1 == 0:                      # b divides a: gcd is the previous remainder
        return x0, y0, d0
    return x1, y1, d1


def inverse(value, modulus, trace=None):
    _, y, g = euclid_ex(modulus, value % modulus, trace)
    if g != 1:
        raise ValueError(f"{value} is not invertible modulo {modulus}: gcd = {g}")
    return y % modulus               # a negative y1 is corrected by adding the modulus
```

Быстрое возведение в степень (листинг 3.3) также реализовано строго по пособию. Встроенная функция `pow(a, z, n)` языка Python не используется, но служит эталоном при самопроверке: на 2000 случайных тройках с 30-значными числами результаты совпали.

Листинг 3.3 – Быстрое возведение в степень по модулю

```
def fast_exp(a, z, n, trace=None):
    a1, z1, x = a % n, z, 1 % n
    while z1 != 0:
        while z1 % 2 == 0:
            z1 //= 2
            a1 = a1 * a1 % n
        z1 -= 1
        x = x * a1 % n
        if trace is not None:
            trace.append(ExpStep(z1, a1, x))
    return x
```

Вычисление ключей (листинг 3.4) проверяет все условия алгоритма: простоту $p$ и $q$ (тестом Миллера–Рабина), их различие, диапазон $e$ и его взаимную простоту с $\varphi(r)$. Нарушение любого условия приводит к понятному сообщению об ошибке, а не к ключу, на котором дешифрование молча даст неверный результат.

Листинг 3.4 – Вычисление пары ключей

```
def make_keys(p, q, e, trace=None):
    for name, value in (("p", p), ("q", q)):
        if not is_prime(value):
            raise ValueError(f"{name} = {value} is not prime")
    if p == q:
        raise ValueError("p and q must be different")
    phi = (p - 1) * (q - 1)
    if not 1 < e < phi:
        raise ValueError(f"e must satisfy 1 < e < phi(r) = {phi}")
    if gcd(e, phi) != 1:
        raise ValueError(f"e = {e} and phi(r) = {phi} are not coprime")
    return Keys(p, q, e, inverse(e, phi, trace))


def encrypt(blocks, e, r):
    _check_blocks(blocks, r)
    return [fast_exp(m, e, r) for m in blocks]


def decrypt(blocks, d, r):
    _check_blocks(blocks, r)
    return [fast_exp(c, d, r) for c in blocks]
```

Текст переводится в блоки одним из двух способов. Кодировка `letters` соответствует пособию: $A = 2, \ldots, Z = 27$, русские буквы (33, включая Ё) продолжают таблицу: А $= 28, \ldots,$ Я $= 60$; пробелы и знаки препинания отбрасываются. Кодировка `unicode` использует коды символов Unicode и позволяет шифровать произвольный текст, но требует модуля $r > 1103$ для кириллицы. Если блок не меньше $r$, программа сообщает об ошибке: такой блок нельзя восстановить, поскольку дешифрование возвращает остаток по модулю $r$.

### 3.3 Интерфейс командной строки

Команда `rsa` работает в трёх режимах (листинг 3.5). Ключи задаются либо через $p$, $q$, $e$ (тогда $d$ вычисляется), либо напрямую парой $(e, r)$ для шифрования и $(d, r)$ для дешифрования — так, как их получает собеседник.

Листинг 3.5 – Примеры запуска

```
python3 main.py rsa keys -p 3 -q 11 -e 7     # r, phi(r), ход алгоритма Евклида, d
python3 main.py rsa enc -t CAT                # 16 29 21 ключи( из пособия)
python3 main.py rsa dec -d 3 -r 33 -t "16 29 21"            # CAT
python3 main.py rsa enc -p 61 -q 53 -e 17 -t "Привет"       # 678 1086 1350 ...
echo "Hi, мир!" | python3 main.py rsa enc --codec unicode -p 1009 -q 1013 -e 17
python3 main.py demo 2                        # все примеры с проверкой
python3 main.py gui 2                         # графическое окно
```

### 3.4 Графическое приложение

Окно лабораторной работы №2 (`python3 main.py gui 2`) содержит поля $p$, $q$, $e$, кнопку вычисления ключей, поля открытого текста и шифротекста, выбор кодировки и панель «Как это работает». При вычислении ключей на панели выводятся $r$, $\varphi(r)$ и таблица расширенного алгоритма Евклида, при шифровании или дешифровании — блоки до и после преобразования и ход быстрого возведения в степень для первого блока. Интерфейс переключается между английским и русским языками.

Общая часть окна (поля ключей, панель разбора, переключатель языка) оформлена базовым классом `KeyWindow`, который переиспользуется окном лабораторной работы №3.

## 4 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

Команда `python3 main.py demo 2` выполняет пример из пособия, шифрование русского текста и произвольного текста с 256-битным модулем, проверку отклонения неверных параметров и самопроверку. Сводные результаты приведены в таблице 4.1, полный вывод — в листинге 4.1.

Таблица 4.1 – Результаты работы программы

| Проверка | Результат | Эталон |
|---|---|---|
| Секретная экспонента для $p = 3, q = 11, e = 7$ | $d = 3$ | совпадает |
| Шифрование «CAT» | 16 29 21 | совпадает |
| Дешифрование 16 29 21 | CAT | совпадает |
| Русский текст, $r = 3233$, $d = 2753$ | восстановлен | — |
| Unicode-текст, 256-битный модуль | восстановлен | — |
| `fast_exp` против `pow`, 2000 троек | совпадение | — |
| 50 случайных пар ключей по 20 блоков | восстановлены | — |

Неверные параметры отклоняются с пояснением: $e = 5$ не взаимно просто с $\varphi(33) = 20$, число 4 не является простым, $e = 25$ выходит за границу $\varphi(r)$, а русские буквы при $r = 33$ не помещаются в диапазон блоков.

Листинг 4.1 – Вывод программы

```
=== Lab 2: RSA cryptosystem ===

=== Example from the lab manual ===

p = 3, q = 11, r = p*q = 33, phi(r) = 20, e = 7
Extended Euclid for a = phi(r) = 20, b = e = 7:
step      q        d        x        y
   0             20        1        0
   1              7        0        1
   2      2       6        1       -2
   3      1       1       -1        3
d = 3 (check: e*d mod phi(r) = 1)
Ko = (7, 33), Kc = (3, 33)

d matches the lab manual

--- Encryption of CAT (codec letters) ---
M = CAT
m = 4 2 21
c = 16 29 21
D = CAT
decryption is correct

Ciphertext matches the lab manual

fast_exp(4, 7, 33):
    z1       a1        x
     6        4        4
     2       16       31
     0       25       16

=== Russian text: the modulus must exceed every letter code ===

p = 61, q = 53, r = p*q = 3233, phi(r) = 3120, e = 17
```

```
Extended Euclid for a = phi(r) = 3120, b = e = 17:
step      q        d        x        y
   0            3120        1        0
   1              17        0        1
   2    183        9        1     -183
   3      1        8       -1      184
   4      1        1        2     -367
d = 2753 (check: e*d mod phi(r) = 1)
Ko = (17, 3233), Kc = (2753, 3233)

--- Russian letters (codec letters) ---
M = Защита авторских прав
m = 36 28 54 37 47 28 28 30 47 43 45 46 39 37 50 44 45 28 30
c = 46 1075 1962 1350 501 1075 1075 1726 501 706 1086 2825 1956 1350 538 678 1086 1075 1726
D = ЗАЩИТААВТОРСКИХПРАВ
decryption is correct

=== Any symbols: 128-bit p and q, e = 65537 ===

p = 315427363538781756665500987021625025781
q = 262896676093679472347933900654311361191
r = 82924805423338389897214052819514817915331336743115935277657371471556577865171
d = 74709129368381096069260212317857590505495779209675118509057976282995080571873

--- Unicode text (codec unicode) ---
M = Это лабораторная работа по КиОКИ, 2026!
m = 39 blocks
  first: 1069
  last:  33
c = 39 blocks
  first: 729158168559013584002080121745077813463365828793419794138869965002393731755 16
  last:  221586563944550875682306703934107635114580001743914962618485976820591201935 62
D = Это лабораторная работа по КиОКИ, 2026!
decryption is correct

=== Invalid parameters are rejected ===

p = 3, q = 11, e = 5: e = 5 and phi(r) = 20 are not coprime: gcd = 5
p = 4, q = 11, e = 7: p = 4 is not prime
p = 3, q = 11, e = 25: e must satisfy 1 < e < phi(r) = 20
Text ЯБЛОКО with r = 33:
  block 60 is outside 0..r-1 = 0..32: choose larger p and q

Self-check: fast_exp equals pow() on 2000 random triples;
50 random key pairs decrypt 20 random blocks each: OK
```

## 5 АНАЛИЗ РЕЗУЛЬТАТОВ

Пример с русским текстом наглядно показывает главную слабость посимвольного применения RSA: одинаковые буквы дают одинаковые блоки шифротекста. Буква «А» сообщения «Защита авторских прав» четыре раза превратилась в число 1075, буква «В» — дважды в 1726. Фактически получен шифр простой замены с алфавитом из чисел, который вскрывается частотным анализом так же, как шифр Цезаря из лабораторной работы №1, несмотря на стойкость самого RSA. Кроме того, при открытом ключе противник может просто зашифровать все 59 возможных букв и составить словарь.

Поэтому на практике RSA применяют иначе:

- несколько символов объединяются в один блок длиной почти $\log_2 r$ бит;
- к блоку добавляется случайное дополнение (схема OAEP), так что одно и то же сообщение каждый раз шифруется по-разному;
- RSA шифрует не само сообщение, а случайный сеансовый ключ симметричного шифра (AES), которым шифруются данные: возведение в степень с 2048-битным модулем в тысячи раз медленнее симметричного шифрования.

Ключи из пособия ($r = 33$, $r = 3233$) пригодны лишь для иллюстрации: модуль такой величины раскладывается перебором делителей до $\sqrt{r}$ за доли миллисекунды, после чего $d$ вычисляется тем же расширенным алгоритмом Евклида. Даже 256-битный модуль из демонстрации разлагается на персональном компьютере за минуты.

# ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучены принципы криптографии с открытым ключом и реализована криптосистема RSA по варианту №1: вычисление пары ключей по заданным $p$, $q$, $e$ расширенным алгоритмом Евклида, шифратор и дешифратор на основе быстрого возведения в степень по модулю. Программа предоставляет интерфейс командной строки и графическое окно с пошаговым разбором вычислений. Корректность подтверждена совпадением с примером пособия ($d = 3$, шифротекст 16 29 21), сравнением `fast_exp` со встроенной функцией `pow` и проверкой обратимости на случайных ключах с модулем до 190 бит.

Практический вывод работы состоит в том, что стойкость алгоритма не переносится автоматически на способ его применения: посимвольное шифрование превращает RSA в шифр простой замены. Надёжная схема требует блоков максимальной длины, случайного дополнения и модуля не менее 2048 бит, а для больших объёмов данных — гибридной схемы, где RSA передаёт только ключ симметричного шифра.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ


1. Ярмолик, С. В. Криптография и охрана коммерческой информации: метод. пособие по выполнению лаб. работ / С. В. Ярмолик, В. Н. Ярмолик. — Минск: БГУИР, 2010. — 32 с.
2. Rivest, R. A method for obtaining digital signatures and public-key cryptosystems / R. Rivest, A. Shamir, L. Adleman // Communications of the ACM. — 1978. — Vol. 21, No. 2. — P. 120–126.
3. Шнайер, Б. Прикладная криптография. Протоколы, алгоритмы, исходные тексты на языке Си / Б. Шнайер. — М.: Триумф, 2002. — 816 с.
4. Столлингс, В. Криптография и защита сетей: принципы и практика / В. Столлингс. — М.: Вильямс, 2001. — 672 с.